

ECE264-003 Fall 2019 Final Exam
8-10 AM, Thursday, Dec 12th, 2019

1. Bitwise Operation

Consider the following program which takes multiple inputs in hexadecimal format. The program takes one bit (from right to left) from each hexadecimal input and produce a combined output in the hexadecimal format. If the length of input is not a multiple of four, zeros are added at the beginning of the first digit of the hexadecimal output.

For example, if the input is:

a2 (binary 1010 0010)

57 (binary 0101 0111)

ba (binary 1011 1010)

5a (binary 0101 1010)

50 (binary 0101 0000)

The output is

1a (binary 0001 1010)

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.
4.  int main(int argc, char ** argv)
5.  {
6.      if (argc < 2)
7.      {
8.          return EXIT_FAILURE;
9.      }
10.
11.     unsigned char oneBitMask;
12.     unsigned char oneBit;
13.     unsigned char inHex;
14.     unsigned char destHex = 0;
15.     for (int i = 1; i < argc; i++)
16.     {
17.         inHex = (unsigned char) strtol(argv[i], NULL, 16);
18.
```

```

19.      // 1.A: Please fix the code
20.      // oneBitMask is the binary mask that takes the i-th
21.      // bit (from right to left) of inHex
22.      // for example, oneBitMask is binary 0001 if i is 1
23.      oneBitMask = 1 << 1.A;
24.
25.      // 1.B: Please fix the code
26.      // oneBit is the i-th bit (from right to left) of inHex
27.      oneBit = inHex 1.B oneBitMask;
28.
29.      // 1.C: Please fix the code
30.      // destHex is the combined output by taking the i-th bit
31.      // (from right to left) of inHex
32.      // hint: you need to use oneBit to answer 1.C
33.      destHex = 1.C;
34.  }
35.
36.      // print in lower-case hexadecimal
37.      printf("%x\n", destHex);
38.      return EXIT_SUCCESS;
39.  }

```

1.A - 1.C: Please fix the code in the above program.

The executable program's name is q1prog.

1.D: what is the output when the program is executed in a terminal using

```
./q1prog a2 6f ec ba 97 58 c2 d1
```

1.E: what is the output when the program is executed in a terminal using

```
./q1prog ac ef 70 98 5 3a d2
```

Answer:

1.A: (i-1)

1.B: &

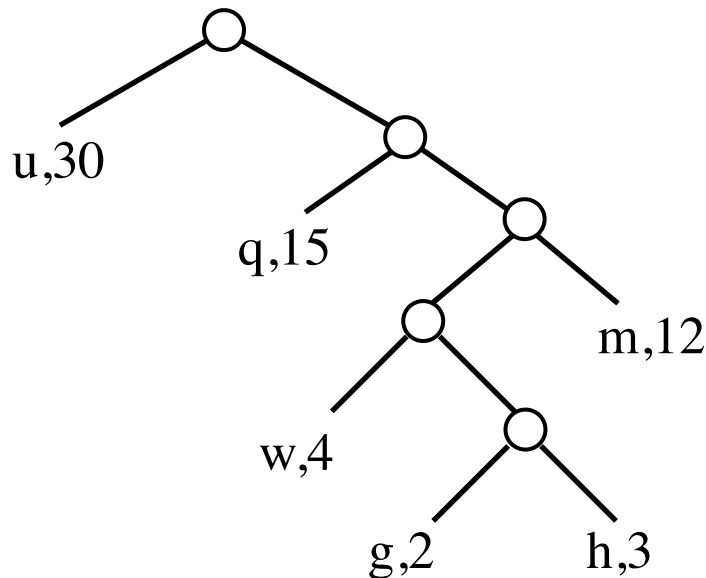
1.C: destHex | oneBit

1.D: de

1.E: 6a

2. Huffman Tree

Consider a file that produces the following Huffman tree to express the code for Huffman compression (the number after comma shows the frequency of the corresponding letter):



2.A: What is the code (binary) of letter q?

2.B: What letter does the code (binary) 11010 stands for?

2.C: How many bytes are needed to store the **data** in the Huffman compressed file?

2.D: We want to change the frequency of letter w and fix the frequencies of all other letters in the file. In order to keep the Huffman tree structure unchanged as above, what is the **lowest** possible frequency of letter w?

2.E: We want to change the frequency of letter w and fix the frequencies of all other letters in the file. In order to keep the Huffman tree structure unchanged as above, what is the **highest** possible frequency of letter w?

Answer:

2.A: 10

2.B: g

2.C: 18

2.D: 3

2.E: 4

3. Huffman Code Book

Consider a file that contains the following letters with the corresponding frequencies.

Letter	P	R	O	G	r	a	m
Frequency	3	5	6	10	27	30	60

We compress this file via Huffman coding and the code for letter r is 111.

The beginning of the **data** of this Huffman compressed file is shown below (output of `xxd` command)

```
db7c 2cfa ...
```

Here ... means more to come. For your reference, the output of `xxd -b` command for the **data** of the same Huffman compressed file is shown below.

```
11011011 01111100 00101100 11111010 ...
```

Please write down the first, third, fifth, seventh, and ninth letters in the beginning of the original file for answers 3.A - 3.E.

Answer:

3.A: P

3.B: G

3.C: m

3.D: G

3.E: O

m 0

a 10

r 111

G 1100

O 11010

P 110110

R 110111

The beginning of the original file is:

PRGmmaGrO...

4. Huffman Compression

Suppose the beginning of the **header** of a Huffman compressed file is (output of `xxd` command)

```
e5e1 7c66 319d b9c0 ...
```

Here ... means more to come. For your reference, the output of `xxd -b` command for the **header** of the same Huffman compressed file is shown below (empty line is added for clarity).

```
11100101 11100001 01111100 01100110 00110001 10011101  
10111001 11000000 ...
```

When the control bit is 1, it is followed by **7** bits of data as a letter.

Please answer the following questions:

4.A: How many different letters are there in the original data file?

4.B: What is the Huffman code (binary) for letter F?

4.C: What is the **letter** with the shortest Huffman code (binary)? If there are more than one letter with shortest code, please answer them all.

4.D: What is **length** of the longest Huffman code (binary)?

4.E: Is the Huffman tree for this compressed file a full binary tree? (Hint: a full binary tree is a binary tree where each internal node has exactly two children).

Please answer Y or N in 4.E.

Answer:

4.A: 7

4.B: 101

4.C: e, a

4.D: 4

4.E: Y

header: 1e1a01x1L01F01m1N000

ASCII Table

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	NUL	32	20	SPACE	64	40	@	96	60	`
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	\$	68	44	D	100	64	d
5	5	ENQ	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	TAB	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL